



Binary Search

Lower/Upper Bound & Busca Binária na Resposta

Profa Gláucya Boechat
glaucya.boechat@ba.docente.senai.br
Slides adaptados dos prof Rubisley Lemes



Lower Bound (Limite inferior)



Situação Problema:

Dado um array ordenado de números inteiros, encontre o **primeiro valor** que seja **maior ou igual** à um **valor alvo**.

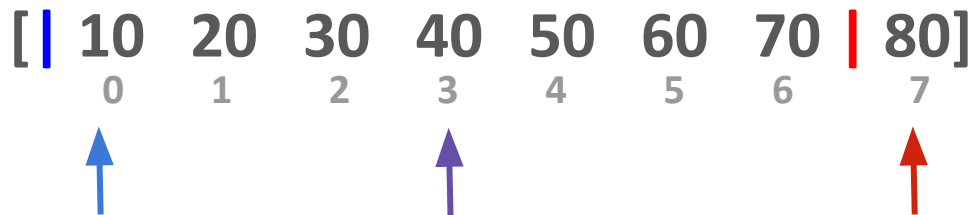
Exemplo:

```
array = [10, 20, 30, 40, 50, 60, 70, 80];  
target_value = 35;
```

Resposta: 40

Algoritmo:

Se `array[mid]` for maior ou igual ao alvo, mova o ponteiro `right` para `mid`, caso contrário, mova o ponteiro `left` para `mid + 1` — enquanto `left < right`.



target_value = 35;

Algoritmo:

Se `array[mid]` for maior ou igual ao alvo, mova o ponteiro `right` para `mid`, caso contrário, mova o ponteiro `left` para `mid + 1` — enquanto `left < right`.

[| 10 20 30 40 50 60 70 | 80]

[| 10 20 30 | 40 50 60 70 80]

0 1 2 3 4 5 6 7



target_value = 35;

Algoritmo:

Se `array[mid]` for maior ou igual ao alvo, mova o ponteiro `right` para `mid`, caso contrário, mova o ponteiro `left` para `mid + 1` — enquanto `left < right`.

[| 10 20 30 40 50 60 70 | 80]

[| 10 20 30 | 40 50 60 70 80]

[10 20 | 30 | 40 50 60 70 80]
0 1 2 3 4 5 6 7



target_value = 35;

Algoritmo:

Se `array[mid]` for maior ou igual ao alvo, mova o ponteiro `right` para `mid`, caso contrário, mova o ponteiro `left` para `mid + 1` — enquanto `left < right`.

[| 10 20 30 40 50 60 70 | 80]

[| 10 20 30 | 40 50 60 70 80]

[10 20 | 30 | 40 50 60 70 80]

[10 20 30 | | 40 50 60 70 80]
0 1 2 3 4 5 6 7

target_value = 35;



Exemplo de lower bound em C++ (1º elemento $\geq$ target)

```
#include <iostream>
#include <vector>
#include <algorithm> // Necessário para lower_bound
using namespace std;

void main() {
    vector<int> array = {10, 20, 30, 40, 50, 60, 70, 80};
    int target_value = 35;
    // lower_bound retorna um iterador para o primeiro elemento  $\geq$  target
    auto it = lower_bound(array.begin(), array.end(), target_value);
    int index = it - array.begin();
    if (it != array.end()) {
        cout << "Lower bound : " << *it << " \n indice : " << index << endl;
    } else {
        cout << "Nenhum elemento encontrado  $\geq$  " << target_value << endl;
    }
}
```

Saída

```
$ ./a.out
Lower bound : 40
indice : 3
```




Upper Bound (Limite superior)



Situação Problema:

Dado um array ordenado de números inteiros, encontre o **primeiro valor** que seja **estritamente maior** à um **valor alvo**.

Exemplo:

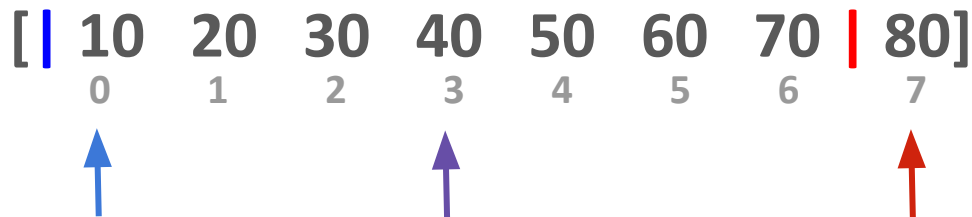
array = [10, 20, 30, 40, 50, 60, 70, 80];

target_value = 40;

Resposta: 50

Algoritmo:

Se `array[mid]` for maior que o alvo, mova o ponteiro **right** para **mid**, caso contrário, mova o ponteiro **left** para **mid + 1** — enquanto **left** < **right**.



Algoritmo:

Se `array[mid]` for maior que o alvo, mova o ponteiro **right** para **mid**, caso contrário, mova o ponteiro **left** para **mid + 1** — enquanto **left** < **right**.

[| 10 20 30 40 50 60 70 | 80]

[10 20 30 40 | 50 60 70 | 80]
0 1 2 3 4 5 6 7



Algoritmo:

Se `array[mid]` for maior que o alvo, mova o ponteiro **right** para **mid**, caso contrário, mova o ponteiro **left** para **mid + 1** — enquanto **left** < **right**.

[| 10 20 30 40 50 60 70 | 80]

[10 20 30 40 | 50 60 70 | 80]

[10 20 30 40 | 50 | 60 70 80]

0

1

2

3

4

5

6

7



Algoritmo:

Se `array[mid]` for maior que o alvo, mova o ponteiro **right** para **mid**, caso contrário, mova o ponteiro **left** para **mid + 1** — enquanto **left** < **right**.

[| 10 20 30 40 50 60 70 | 80]

[10 20 30 40 | 50 60 70 | 80]

[10 20 30 40 | 50 | 60 70 80]

[10 20 30 40 | | 50 60 70 80]
0 1 2 3 4 5 6 7



Exemplo de upper bound em C++ (1º elemento > target)

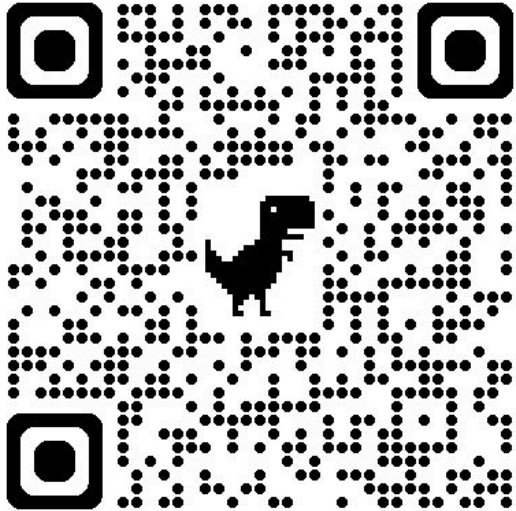
```
#include <iostream>
#include <vector>
#include <algorithm> // Necessário para upper_bound
using namespace std;

void main() {
    vector<int> array = {10, 20, 30, 40, 50, 60, 70, 80};
    int target_value = 40;
    // upper_bound retorna o primeiro elemento > target_value
    auto it = upper_bound(array.begin(), array.end(), target_value);
    int index = it - array.begin();
    if (it != array.end()) {
        cout << "Upper bound value: " << *it << " \n indice : " << index <<
endl;
    } else {
        cout << "Nenhum elemento encontrado > " << target_value << endl;
    }
}
```

Saída

```
$ ./a.out
Upper bound value: 50
indice : 4
```

Time to code :)



<https://codeforces.com/problemset/problem/706/B/>

B. Interesting drink

time limit per test: 2 seconds

memory limit per test: 256 megabytes

Vasily likes to rest after a hard work, so you may often meet him in some bar nearby. As all programmers do, he loves the famous drink "Beecola", which can be bought in n different shops in the city. It's known that the price of one bottle in the shop i is equal to x_i coins.

Vasily plans to buy his favorite drink for q consecutive days. He knows, that on the i -th day he will be able to spent m_i coins. Now, for each of the days he want to know in how many different shops he can buy a bottle of "Beecola".

Input

The first line of the input contains a single integer n ($1 \leq n \leq 100\,000$) — the number of shops in the city that sell Vasily's favourite drink.

The second line contains n integers x_i ($1 \leq x_i \leq 100\,000$) — prices of the bottles of the drink in the i -th shop.

The third line contains a single integer q ($1 \leq q \leq 100\,000$) — the number of days Vasily plans to buy the drink.

Then follow q lines each containing one integer m_i ($1 \leq m_i \leq 10^9$) — the number of coins Vasily can spent on the i -th day.

Output

Print q integers. The i -th of them should be equal to the number of shops where Vasily will be able to buy a bottle of the drink on the i -th day.

Example

input

```
5
3 10 8 6 11
4
1
10
3
11
```

[Copy](#)

output

```
0
4
1
5
```

[Copy](#)

Codeforces Round 367 (Div. 2)

Finished

Virtual participation

Virtual contest is a way to take part in past contest, as close as possible to participation on time. It is supported only ICPC mode for virtual contests. If you've seen these problems, a virtual contest is not for you - solve these problems in the archive. If you just want to solve some problem from a contest, a virtual contest is not for you - solve this problem in the archive. Never use someone else's code, read the tutorials or communicate with other person during a virtual contest.

[Start virtual contest](#)

Problem tags

[binary search](#) [dp](#) [implementation](#) [*1100](#)

No tag edit access

Contest materials

- Announcement (en) [✕](#)
- Tutorial (en) [✕](#)

Busca Binária na Resposta

(em funções arbitrárias monótonas)

Às vezes nos deparamos com problemas de buscar o **valor mínimo** ou **valor máximo** em uma função arbitrária.

- O que nos impede de testar todos os valores possíveis até encontrar a resposta?

Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

Podemos testar todos os valores possíveis de n até encontrar o menor que satisfaz.

Obs: sim, dá pra resolver por *delta* e *bhaskara*, mas não vamos por esse caminho

Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

Vamos testar todos os valores possíveis 1 até S , pois $n \leq S$:

$$f(1) = 1 \mid f(2) = 3 \mid f(3) = 6 \mid f(4) = 10 \mid f(5) = 15 \mid f(6) = 21 \mid f(7) = 28 \mid \dots \mid f(17) = 153$$

Achamos que **o menor n é 6**. Mas qual o problema dessa abordagem?

Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

Vamos testar todos os valores possíveis 1 até S , pois $n \leq S$:

$f(1) = 1$ | $f(2) = 3$ | $f(3) = 6$ | $f(4) = 10$ | $f(5) = 15$ | **$f(6) = 21$** | $f(7) = 28$ | ... | $f(17) = 153$

Achamos que **o menor n é 6**. Mas qual o problema dessa abordagem?

Complexidade: $O(n)$

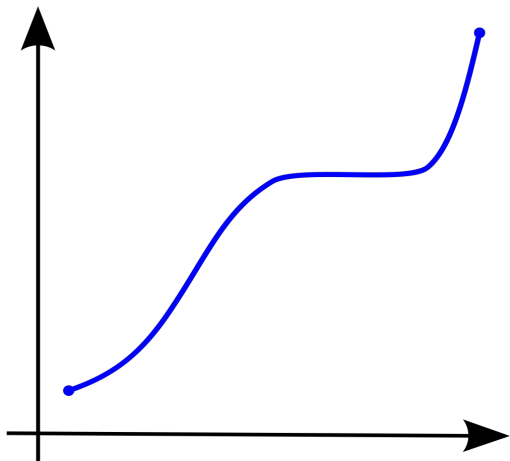
Para valores de S maiores que 10^8 , o algoritmo é lento.

Funções monótonas

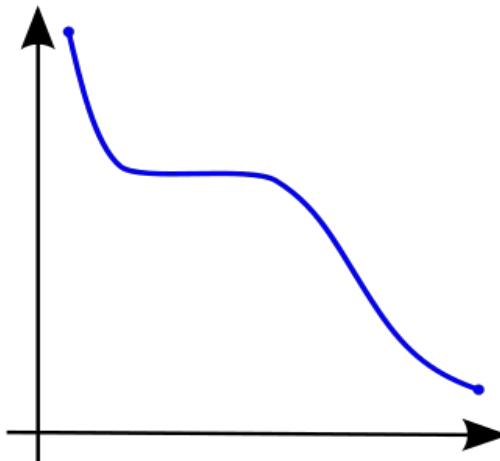
É uma função entre dois conjuntos ordenados que preserva ou inverte a relação de ordem.

- f é **monótona não-decrescente** quando $\forall x, y \in A, (x, y \Rightarrow f(x) \geq f(y))$
- f é **monótona não-crescente** quando $\forall x, y \in A, (x, y \Rightarrow f(x) \leq f(y))$

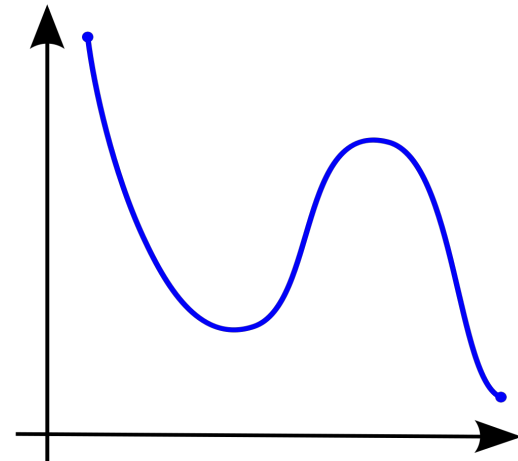
monótona não-decrescente



monótona não-crescente



não monótona



Suponha que temos uma função booleana ***ok(x)***.

Normalmente, em problemas desse tipo, queremos encontrar o valor máximo ou mínimo de ***x*** tal que ***ok(x)*** seja verdadeiro.

Da mesma forma que a busca binária em um array só funciona se o array estiver ordenado, a busca binária sobre a resposta só funciona se a função resposta for *monótona*, ou seja, sempre não decrescente ou sempre não crescente.

x	0	1	...	$k-1$	k	$k+1$	...
$ok(x)$	false	false	...	false	true	true	...

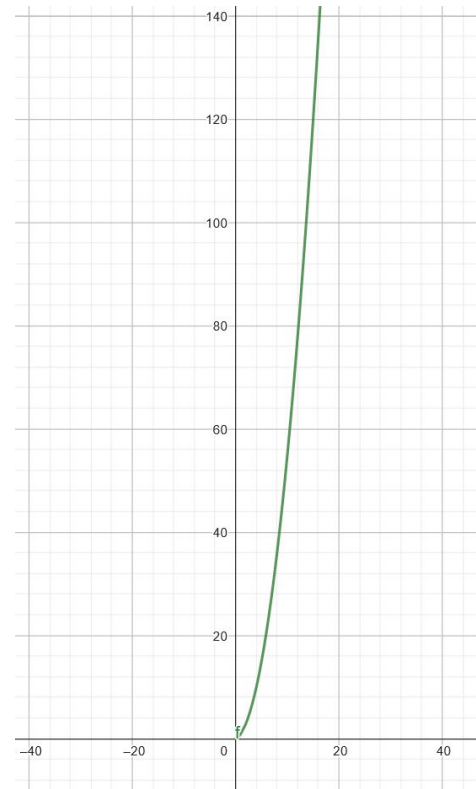
Nesse caso, k é o menor valor possível que satisfaz a função $ok(x)$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor **n** inteiro que satisfaz essa função?

$$\frac{n(n + 1)}{2} \geq 17$$

Vamos aplicar busca binária na resposta no espaço de 1 até S , pois $f(n)$ é uma função monótona não-decrescente!



Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$

/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L								M									R

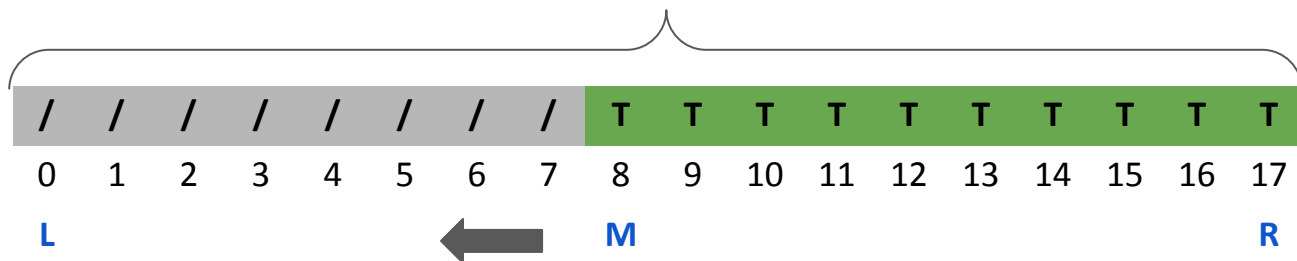
$ok(M) = ?$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$



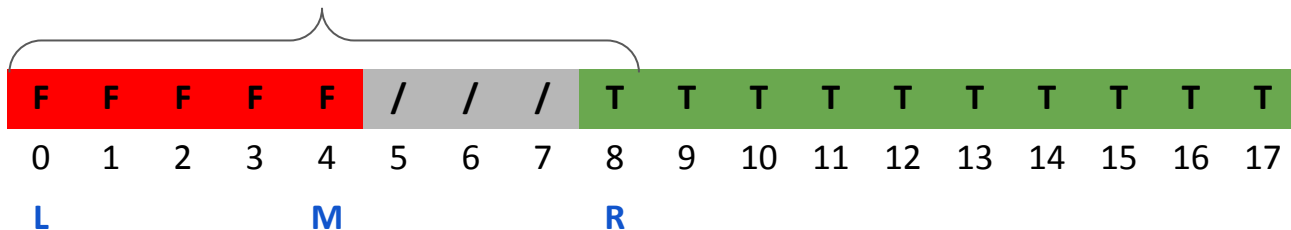
$ok(M) = \text{True},$
 $\text{pois } f(8) = 36 > 17$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$



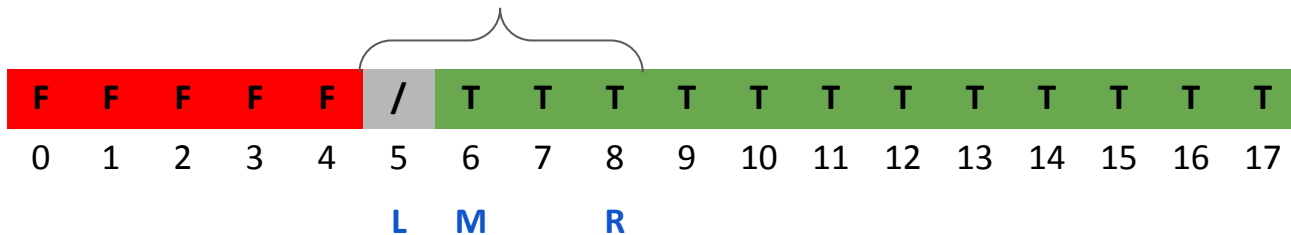
$ok(M) = \text{False},$
 $\text{pois } f(4) = 10 < 17$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$



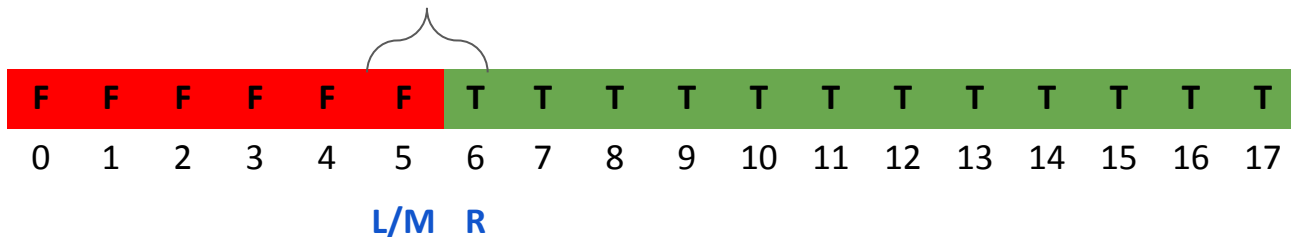
$ok(M) = True,$
 $pois f(6) = 21 > 17$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$



$ok(M) = \text{False},$
 $\text{pois } f(5) = 15 < 17$

Voltando para o Exemplo: $f(n) = S = n * (n + 1) / 2$.

- Se sabemos que $S = 17$, qual o menor n inteiro que satisfaz essa função?

$$\frac{n(n+1)}{2} \geq 17$$

- Espaço de busca: $[0, 17]$
- Função $ok(n) = (f(n) \geq 17)$, $ok(n) \in \{True, False\}$

F	F	F	F	F	F	T	T	T	T	T	T	T	T	T	T	T	T
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

L/R



R = 6

$$f(6) = 21$$

Exemplo Busca Binária na Resposta S = 17

$$\frac{n(n+1)}{2} \geq 17$$

```
#include <iostream>
using namespace std;
int S = 17;

bool ok(int& n) {
    return n * (n + 1) / 2 >= S;
}

int binarySearch() {
    int l = 0, r = S;
    while(l < r){
        int mid = l + (r - l) / 2;
        if(ok(mid))
            r = mid;
        else
            l = mid+1;
    }
    return r;
}
```

```
int main() {
    int result = binarySearch();

    cout << "Valor de S: " << S << endl;
    cout << "Menor n inteiro que satisfaz a função: " <<
        result << endl;
    cout << "f(" << result << "): " <<
        (result * (result + 1) / 2) << endl;
    return 0;
}
```

Saída

```
$ ./a.out
Valor de S: 17
Menor n inteiro que satisfaz a função: 6
f(6): 21
```

- Quando realizamos uma busca binária em uma resposta, começamos com um espaço de busca finito de tamanho **N** , no qual sabemos que a resposta está contida.
- Em cada iteração da busca binária, o espaço de busca é reduzido pela metade, então o algoritmo testa **$O(\log N)$** valores.
- Isso é eficiente e muito melhor do que testar cada valor possível no espaço de busca.

Complexidade de algoritmos de Busca Binária na Resposta

A função `ok()` é executada várias vezes no algoritmo, mais especificamente **$\log M$** vezes, em que M é o tamanho do espaço da resposta.

Então, a complexidade (geralmente) é **$O(x * \log M)$** , em que x é a complexidade da função `ok()`.

No nosso exemplo anterior, a complexidade da função `ok(n)` era $O(1)$. Então a complexidade total do algoritmo foi $O(\log S)$, onde S era o valor de $f(n)$ proposto. Permite valores de entradas grandes, como 10^{18} .

Passos para resolver

1. Definir intervalo de resposta;
2. Definir a função `ok()` — pode ficar difícil em algumas questões;
3. Implementar a busca binária no intervalo definido e com o critério da função `ok()`.

Problema: [Hamburgers](#) (Codeforces)

<https://codeforces.com/contest/371/problem/C>





C. Hamburgers

time limit per test: 1 second

memory limit per test: 256 megabytes

Polycarpus loves hamburgers very much. He especially adores the hamburgers he makes with his own hands. Polycarpus thinks that there are only three decent ingredients to make hamburgers from: a bread, sausage and cheese. He writes down the recipe of his favorite "Le Hamburger de Polycarpus" as a string of letters 'B' (bread), 'S' (sausage) and 'C' (cheese). The ingredients in the recipe go from bottom to top, for example, recipe "BSCBS" represents the hamburger where the ingredients go from bottom to top as bread, sausage, cheese, bread and sausage again.

Polycarpus has n_b pieces of bread, n_s pieces of sausage and n_c pieces of cheese in the kitchen. Besides, the shop nearby has all three ingredients, the prices are p_b rubles for a piece of bread, p_s for a piece of sausage and p_c for a piece of cheese.

Polycarpus has r rubles and he is ready to shop on them. What maximum number of hamburgers can he cook? You can assume that Polycarpus cannot break or slice any of the pieces of bread, sausage or cheese. Besides, the shop has an unlimited number of pieces of each ingredient.

Input

The first line of the input contains a non-empty string that describes the recipe of "Le Hamburger de Polycarpus". The length of the string doesn't exceed 100, the string contains only letters 'B' (uppercase English B), 'S' (uppercase English S) and 'C' (uppercase English C).

The second line contains three integers n_b, n_s, n_c ($1 \leq n_b, n_s, n_c \leq 100$) — the number of the pieces of bread, sausage and cheese on Polycarpus' kitchen. The third line contains three integers p_b, p_s, p_c ($1 \leq p_b, p_s, p_c \leq 100$) — the price of one piece of bread, sausage and cheese in the shop. Finally, the fourth line contains integer r ($1 \leq r \leq 10^{12}$) — the number of rubles Polycarpus has.

Please, do not write the `%lld` specifier to read or write 64-bit integers in C++. It is preferred to use the `cin`, `cout` streams or the `%I64d` specifier.

Output

Print the maximum number of hamburgers Polycarpus can make. If he can't make any hamburger, print 0.

Examples

input

```
BBBSSC
6 4 1
1 2 3
4
```

output

```
2
```

Codeforces Round 218 (Div. 2)

Finished

→ Practice?

Want to solve the contest problems after the official contest ends? Just register for practice and you will be able to submit solutions.

[Register for practice](#)

→ Virtual participation

Virtual contest is a way to take part in past contest, as close as possible to participation on time. It is supported only ICPC mode for virtual contests. If you've seen these problems, a virtual contest is not for you - solve these problems in the archive. If you just want to solve some problem from a contest, a virtual contest is not for you - solve this problem in the archive. Never use someone else's code, read the tutorials or communicate with other person during a virtual contest.

[Start virtual contest](#)

→ Problem tags

[binary search](#) [brute force](#) [*1600](#)

No tag edit access

→ Contest materials

- Announcement ✕
- Tutorial ✕